

Sri Lanka Economic Sentiment Index (SL-ESI) - Complete EDA & Analysis

This notebook provides a comprehensive exploratory data analysis of the Sri Lanka Economic Sentiment Index project, covering:

- News data collection and preprocessing
- Sentiment analysis using FinBERT
- Economic indicators integration
- Correlation analysis
- Key events impact
- Forecasting results
- Key insights and conclusions

Project Overview

This project analyzes the relationship between news media sentiment and Sri Lanka's economic indicators from 2020-2026, covering:

- **Pre-crisis period** (2020-2021)
- **Economic crisis** (2022)
- **Recovery phase** (2023-2026)

Key Objectives

1. Quantify economic sentiment from news articles using NLP
2. Correlate sentiment with macroeconomic indicators
3. Identify leading/lagging relationships
4. Forecast future sentiment trends

```
In [1]: # Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')

# Set styles
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 10
```

1. Data Loading & Overview

```
In [6]: # Processed data
merged_df = pd.read_csv('src/data/processed/sl esi_merged_economic.csv', parse_date
monthly_index = pd.read_csv('src/data/processed/sl esi_monthly_index.csv')
tourism_df = pd.read_csv('src/data/processed/sl tourism_monthly.csv')
inflation_df = pd.read_csv('src/data/processed/sl inflation_rate.csv')
exchange_df = pd.read_csv('src/data/processed/sl_usd_lkr_exchange.csv')
correlation_df = pd.read_csv('src/data/processed/correlation_summary.csv')
forecast_df = pd.read_csv('src/data/processed/sl esi_forecast_results.csv')

# Raw data
news_raw = pd.read_csv('src/data/raw/news_headlines_raw.csv', parse_dates=['date'])
news_clean = pd.read_csv('src/data/processed/news_headlines_clean.csv', parse_dates=

# Reference data
key_events = pd.read_csv('src/data/reference/key_events.csv', parse_dates=['date'])

print(f"\nDataset Summary:")
print(f" - Merged Economic Data: {len(merged_df)} rows")
print(f" - Monthly Index: {len(monthly_index)} rows")
print(f" - News Articles (Raw): {len(news_raw)} rows")
print(f" - News Articles (Clean): {len(news_clean)} rows")
print(f" - Key Events: {len(key_events)} events")
print(f" - Forecast Results: {len(forecast_df)} rows")
```

Dataset Summary:

- Merged Economic Data: 79 rows
- Monthly Index: 79 rows
- News Articles (Raw): 5877 rows
- News Articles (Clean): 5877 rows
- Key Events: 12 events
- Forecast Results: 12 rows

2. News Data Collection & Preprocessing

2.1 Data Collection Statistics

```
In [7]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Articles per year
news_clean['year'] = news_clean['date'].dt.year
yearly_counts = news_clean['year'].value_counts().sort_index()

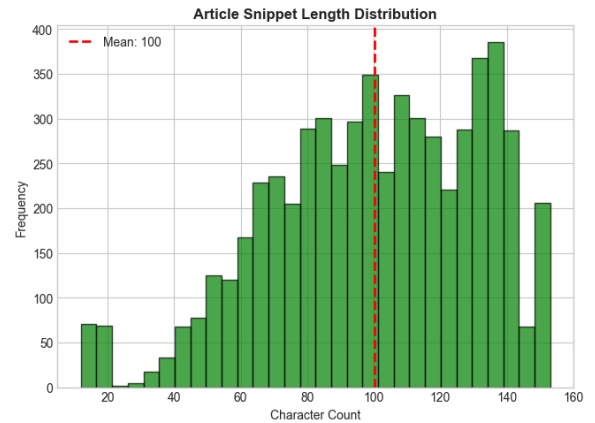
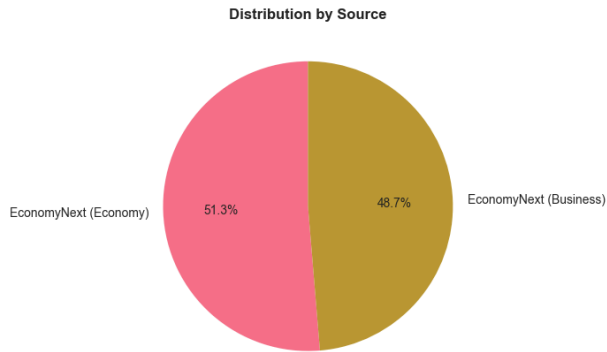
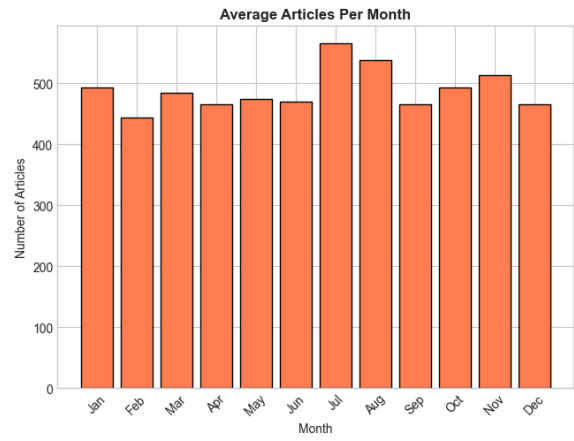
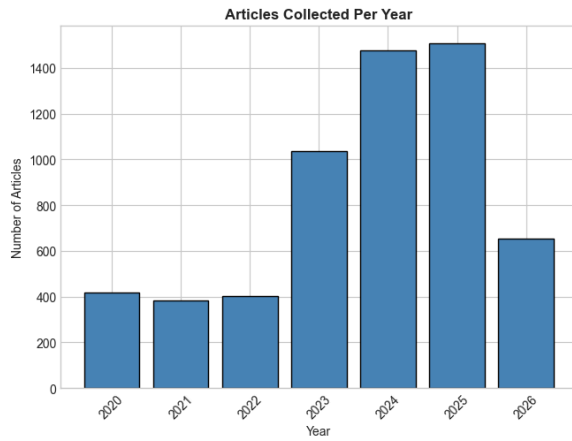
axes[0, 0].bar(yearly_counts.index, yearly_counts.values,
               color='steelblue', edgecolor='black')
axes[0, 0].set_title('Articles Collected Per Year', fontweight='bold')
axes[0, 0].set_xlabel('Year')
axes[0, 0].set_ylabel('Number of Articles')
axes[0, 0].tick_params(axis='x', rotation=45)

# Articles per month (average)
news_clean['month'] = news_clean['date'].dt.month
monthly_avg = news_clean.groupby('month').size()
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
axes[0, 1].bar(months, monthly_avg.values, color='coral', edgecolor='black')
axes[0, 1].set_title('Average Articles Per Month', fontweight='bold')
axes[0, 1].set_xlabel('Month')
axes[0, 1].set_ylabel('Number of Articles')
axes[0, 1].tick_params(axis='x', rotation=45)

# Data sources
if 'source' in news_clean.columns:
    source_counts = news_clean['source'].value_counts()
    axes[1, 0].pie(source_counts.values, labels=source_counts.index,
                  autopct='%1.1f%%', startangle=90)
    axes[1, 0].set_title('Distribution by Source', fontweight='bold')

# Article length distribution (if snippet available)
if 'snippet' in news_clean.columns:
    news_clean['snippet_length'] = news_clean['snippet'].fillna('').str.len()
    axes[1, 1].hist(news_clean['snippet_length'], bins=30,
                   color='green', edgecolor='black', alpha=0.7)
    axes[1, 1].axvline(news_clean['snippet_length'].mean(),
                      color='red', linestyle='--', linewidth=2,
                      label=f'Mean: {news_clean["snippet_length"].mean():.0f}')
    axes[1, 1].set_title('Article Snippet Length Distribution', fontweight='bold')
    axes[1, 1].set_xlabel('Character Count')
    axes[1, 1].set_ylabel('Frequency')
    axes[1, 1].legend()

plt.tight_layout()
plt.show()
```



2.2 Sample Articles

```
In [9]: # Crisis period (2022)
crisis_articles = news_clean[news_clean['date'].dt.year == 2022].sample(3)
print("\n📰 CRISIS PERIOD (2022):")
for idx, row in crisis_articles.iterrows():
    print(f"\nDate: {row['date'].strftime('%Y-%m-%d')}")
    print(f>Title: {row['title'][:100]}...")
    if 'snippet' in row:
        print(f"Snippet: {str(row['snippet'][:150])}...")

# Recovery period (2024-2025)
recovery_articles = news_clean[
    (news_clean['date'].dt.year >= 2024) &
    (news_clean['date'].dt.year <= 2025)
].sample(3)

print("\n📰 RECOVERY PERIOD (2024-2025):")
for idx, row in recovery_articles.iterrows():
    print(f"\nDate: {row['date'].strftime('%Y-%m-%d')}")
    print(f>Title: {row['title'][:100]}...")
    if 'snippet' in row:
        print(f"Snippet: {str(row['snippet'][:150])}...")
```

📰 CRISIS PERIOD (2022):

Date: 2022-08-02

Title: Sri Lanka state sector workers called back to work on Fridays...

Snippet: The decision that was valid for three months has been cancelled with immediate effect. By Neshella Perera August 2, 2022...

Date: 2022-06-07

Title: Sri Lanka to write off small-time farmers' debt 100 percent...

Snippet: "We know that farming families who cultivate paddy on small lands are in a very precarious position." June 7, 2022...

Date: 2022-10-30

Title: Sri Lanka's Ceylon Tea prices fall for the third week...

Snippet: Rupee average rose to as much as 1676.93 rupees a kilo in the first week of October. October 30, 2022...

📈 RECOVERY PERIOD (2024-2025):

Date: 2025-01-13

Title: Sri Lanka central bank revalues restructured bonds booking profit...

Snippet: The central bank's net assets, were barely in the positive since the domestic debt re-structure January 13, 2025...

Date: 2024-10-25

Title: Sri Lanka in talks to end Israel linked travel advisories, full security to tourists: Minister...

Snippet: We have taken action, if required we will do more in the future October 25, 2024...

Date: 2024-06-25

Title: Sri Lanka to sign Paris Club debt deals Wednesday: Minister...

Snippet: The finalizing of the restructure was a 'great victory' for Sri Lanka June 25, 2024...

3. Sentiment Analysis Results

3.1 Sentiment Distribution Over Time

```
In [11]: # Create visualization
fig = make_subplots(rows=2, cols=1,
                    subplot_titles=('Monthly Article Count', 'Sentiment Intensity Over Time'),
                    vertical_spacing=0.12)

# Article count
fig.add_trace(
    go.Bar(x=monthly_index['year_month'], y=monthly_index['article_count'],
           name='Article Count', marker_color='steelblue'),
    row=1, col=1
)

# Sentiment intensity
fig.add_trace(
    go.Scatter(x=monthly_index['year_month'], y=monthly_index['sentiment_intensity']
```

```

        name='Sentiment Intensity', mode='lines+markers',
        line=dict(color='coral', width=3),
        marker=dict(size=6)),
    row=2, col=1
)

# Add crisis period highlight
fig.add_vrect(x0="2022-01", x1="2022-12",
              annotation_text="Economic Crisis",
              annotation_position="top left",
              fillcolor="red", opacity=0.1, line_width=0,
              row=1, col=1)
fig.add_vrect(x0="2022-01", x1="2022-12",
              fillcolor="red", opacity=0.1, line_width=0,
              row=2, col=1)

fig.update_layout(
    height=700,
    showlegend=True,
    template='plotly_white',
    title_text="SL-ESI: News Volume & Sentiment Trends (2020-2026)",
    title_font_size=16
)

fig.update_xaxes(title_text="Date", row=2, col=1)
fig.update_yaxes(title_text="Number of Articles", row=1, col=1)
fig.update_yaxes(title_text="Sentiment Intensity", row=2, col=1)

fig.show()

```

3.2 Sentiment Statistics

```

In [12]: print(f"\n Overall Statistics (2020-2026):")
print(f"  Total Articles Analyzed: {monthly_index['article_count'].sum():,}")
print(f"  Time Period: {monthly_index['year_month'].min()} to {monthly_index['year_
print(f"  Average Articles/Month: {monthly_index['article_count'].mean():.1f}")

print(f"\n Sentiment Metrics:")
print(f"  Mean Sentiment: {monthly_index['mean_sentiment'].mean():.4f}")
print(f"  Std Sentiment: {monthly_index['mean_sentiment'].std():.4f}")
print(f"  Min Sentiment: {monthly_index['mean_sentiment'].min():.4f}")
print(f"  Max Sentiment: {monthly_index['mean_sentiment'].max():.4f}")

print(f"\n Sentiment Intensity:")
print(f"  Mean Intensity: {monthly_index['sentiment_intensity'].mean():.4f}")
print(f"  Std Intensity: {monthly_index['sentiment_intensity'].std():.4f}")
print(f"  Min Intensity: {monthly_index['sentiment_intensity'].min():.4f}")
print(f"  Max Intensity: {monthly_index['sentiment_intensity'].max():.4f}")

# By year
print(f"\n Year-wise Breakdown:")
monthly_index['year'] = pd.to_datetime(monthly_index['year_month']).dt.year
yearly_stats = monthly_index.groupby('year').agg({
    'article_count': 'sum',
    'mean_sentiment': 'mean',

```

```

        'sentiment_intensity': 'mean'
    }).round(4)

print(yearly_stats.to_string())

```

Overall Statistics (2020-2026):
 Total Articles Analyzed: 5,877
 Time Period: 2020-02 to 2026-08
 Average Articles/Month: 74.4

Sentiment Metrics:
 Mean Sentiment: 0.0326
 Std Sentiment: 0.1464
 Min Sentiment: -0.4516
 Max Sentiment: 0.3559

Sentiment Intensity:
 Mean Intensity: 0.1687
 Std Intensity: 0.5612
 Min Intensity: -1.3053
 Max Intensity: 1.3915

Year-wise Breakdown:

	article_count	mean_sentiment	sentiment_intensity
year			
2020	417	-0.0027	-0.0611
2021	383	0.0208	0.0706
2022	403	-0.1078	-0.3437
2023	1036	0.0027	0.0325
2024	1477	0.0895	0.4288
2025	1508	0.1041	0.5012
2026	653	0.1621	0.7158

3.3 Most Positive & Negative Months

```

In [14]: # Top 5 positive months
top_positive = monthly_index.nlargest(5, 'sentiment_intensity')
print("TOP 5 MOST POSITIVE MONTHS")
for idx, row in top_positive.iterrows():
    print(f"\n{row['year_month']}")
    print(f"  Articles: {row['article_count']}")
    print(f"  Mean Sentiment: {row['mean_sentiment']:.4f}")
    print(f"  Sentiment Intensity: {row['sentiment_intensity']:.4f}")

print("\n" + "="*80)

# Top 5 negative months
top_negative = monthly_index.nsmallest(5, 'sentiment_intensity')
print("TOP 5 MOST NEGATIVE MONTHS")
for idx, row in top_negative.iterrows():
    print(f"\n{row['year_month']}")
    print(f"  Articles: {row['article_count']}")
    print(f"  Mean Sentiment: {row['mean_sentiment']:.4f}")
    print(f"  Sentiment Intensity: {row['sentiment_intensity']:.4f}")

```

TOP 5 MOST POSITIVE MONTHS

2026-07

Articles: 82
Mean Sentiment: 0.3149
Sentiment Intensity: 1.3915

2020-08

Articles: 40
Mean Sentiment: 0.3559
Sentiment Intensity: 1.3215

2026-08

Articles: 75
Mean Sentiment: 0.2794
Sentiment Intensity: 1.2101

2025-08

Articles: 111
Mean Sentiment: 0.2333
Sentiment Intensity: 1.1009

2025-02

Articles: 125
Mean Sentiment: 0.1989
Sentiment Intensity: 0.9621

=====

TOP 5 MOST NEGATIVE MONTHS

2022-05

Articles: 17
Mean Sentiment: -0.4516
Sentiment Intensity: -1.3053

2022-11

Articles: 26
Mean Sentiment: -0.3502
Sentiment Intensity: -1.1541

2020-04

Articles: 45
Mean Sentiment: -0.2511
Sentiment Intensity: -0.9614

2022-06

Articles: 44
Mean Sentiment: -0.2309
Sentiment Intensity: -0.8789

2020-10

Articles: 46
Mean Sentiment: -0.2088
Sentiment Intensity: -0.8038

4. Economic Indicators Overview

4.1 Tourism Arrivals

```
In [19]: fig, ax = plt.subplots(figsize=(14, 6))

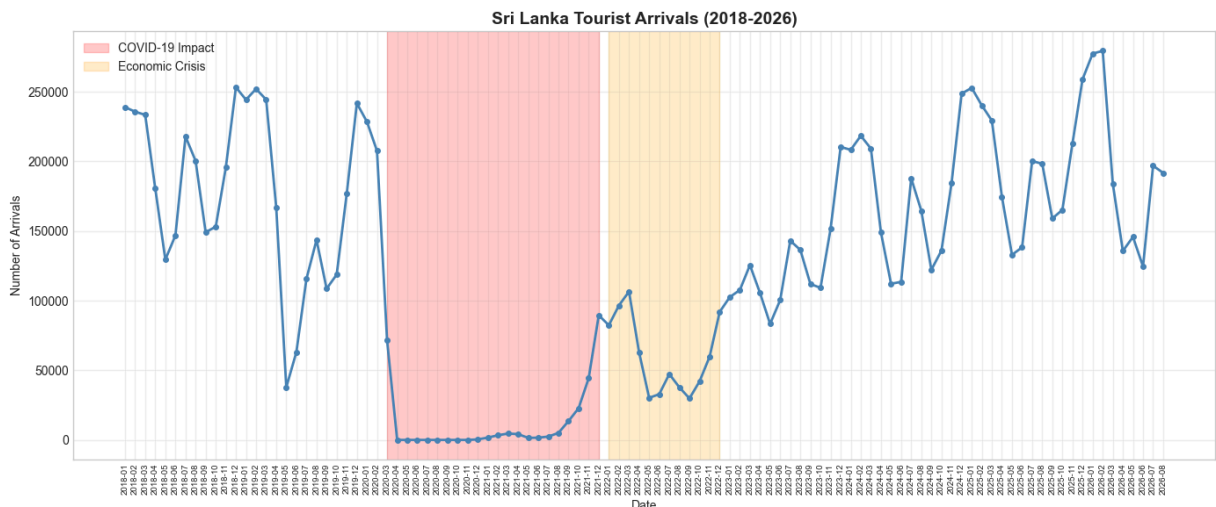
# Plot tourism data
ax.plot(tourism_df['year_month'], tourism_df['tourist_arrivals'],
        marker='o', linewidth=2, markersize=4, color='steelblue')

# Highlight crisis period
ax.axvspan('2020-03', '2021-12', alpha=0.2, color='red',
          label='COVID-19 Impact')
ax.axvspan('2022-01', '2022-12', alpha=0.2, color='orange',
          label='Economic Crisis')

ax.set_title('Sri Lanka Tourist Arrivals (2018-2026)', fontweight='bold', fontsize=
ax.set_xlabel('Date')
ax.set_ylabel('Number of Arrivals')
ax.legend()
ax.tick_params(axis='x', rotation=90)
ax.tick_params(axis='x', labelsizes=7)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nPre-Crisis (2019) Average: {tourism_df[tourism_df['year_month'] < '2020']}
print(f"Crisis Period (2022) Average: {tourism_df[(tourism_df['year_month'] >= '202
print(f"Recovery (2024) Average: {tourism_df[tourism_df['year_month'].str.startswit
print(f"Latest (2026) Average: {tourism_df[tourism_df['year_month'].str.startswith(
```



Pre-Crisis (2019) Average: 176,979
Crisis Period (2022) Average: 59,998
Recovery (2024) Average: 171,122
Latest (2026) Average: 191,890

4.2 Exchange Rate (USD/LKR)

```
In [21]: fig, ax = plt.subplots(figsize=(14, 6))

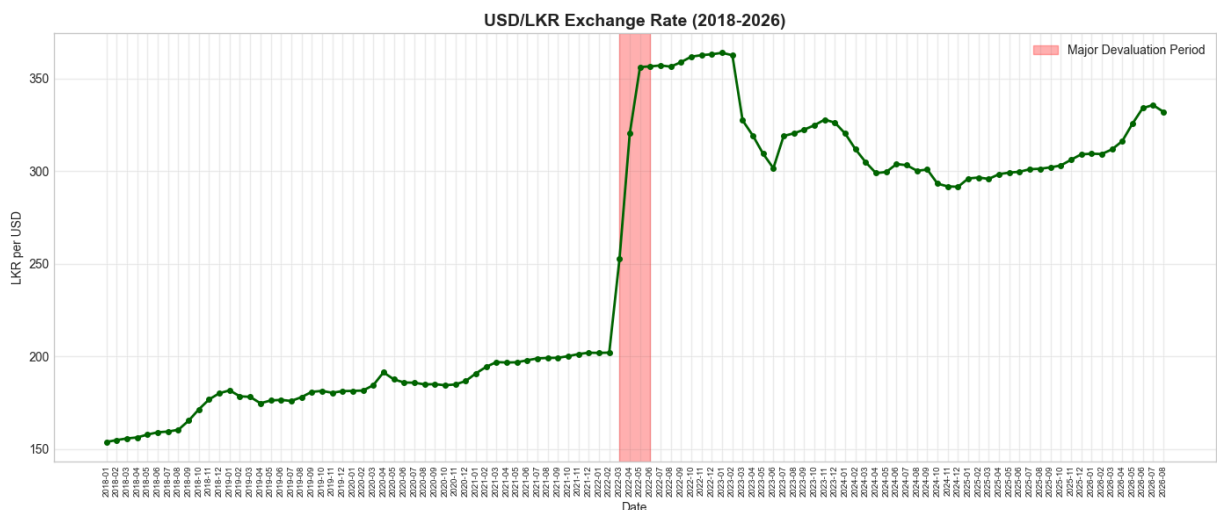
ax.plot(exchange_df['year_month'], exchange_df['usd_lkr_rate'],
        marker='o', linewidth=2, markersize=4, color='darkgreen')

# Highlight major devaluation
ax.axvspan('2022-03', '2022-06', alpha=0.3, color='red',
          label='Major Devaluation Period')

ax.set_title('USD/LKR Exchange Rate (2018-2026)', fontweight='bold', fontsize=14)
ax.set_xlabel('Date')
ax.set_ylabel('LKR per USD')
ax.legend()
ax.tick_params(axis='x', rotation=90)
ax.tick_params(axis='x', labelsiz=7)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Exchange rate statistics
print(f"\nPre-Crisis (2021) Average: {exchange_df[exchange_df['year_month'].str.startswith('2021')]['usd_lkr_rate'].mean()}")
print(f"Crisis Peak (2022) Average: {exchange_df[exchange_df['year_month'].str.startswith('2022')]['usd_lkr_rate'].mean()}")
print(f"Current (2026) Average: {exchange_df[exchange_df['year_month'].str.startswith('2026')]['usd_lkr_rate'].mean()}")
print(f"\nDepreciation from 2021 to 2022: {((exchange_df[exchange_df['year_month'].str.startswith('2022')]['usd_lkr_rate'].mean() - exchange_df[exchange_df['year_month'].str.startswith('2021')]['usd_lkr_rate'].mean()) / exchange_df[exchange_df['year_month'].str.startswith('2021')]['usd_lkr_rate'].mean()) * 100}%")
```



Pre-Crisis (2021) Average: 197.81
 Crisis Peak (2022) Average: 320.84
 Current (2026) Average: 321.81

Depreciation from 2021 to 2022: 62.2%

4.3 Inflation Rate

```
In [23]: fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(inflation_df['year_month'], inflation_df['inflation_rate'],
        marker='o', linewidth=2, markersize=4, color='darkred')

# Highlight hyperinflation
```

```

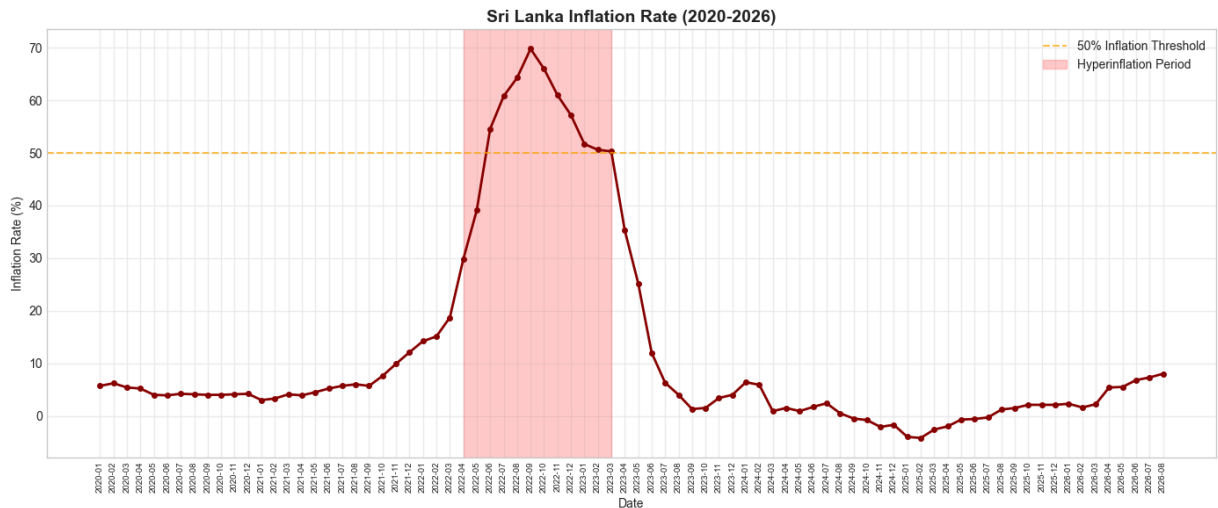
ax.axhline(y=50, color='orange', linestyle='--', alpha=0.7,
           label='50% Inflation Threshold')
ax.axvspan('2022-04', '2023-03', alpha=0.2, color='red',
           label='Hyperinflation Period')

ax.set_title('Sri Lanka Inflation Rate (2020-2026)', fontweight='bold', fontsize=14)
ax.set_xlabel('Date')
ax.set_ylabel('Inflation Rate (%)')
ax.legend()
ax.tick_params(axis='x', rotation=90)
ax.tick_params(axis='x', labelsizes=7)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Inflation statistics
print(f"\nPre-Crisis (2021) Average: {inflation_df[inflation_df['year_month'].str.startswith('2021')].inflation_rate.mean():.2%}")
print(f"Crisis Peak (2022) Average: {inflation_df[inflation_df['year_month'].str.startswith('2022')].inflation_rate.mean():.2%}")
print(f"Peak Month: {inflation_df.loc[inflation_df['inflation_rate'].idxmax(), 'year_month'].strftime('%Y-%m')}")
print(f"Current (2026) Average: {inflation_df[inflation_df['year_month'].str.startswith('2026')].inflation_rate.mean():.2%}")

```



Pre-Crisis (2021) Average: 5.92%
 Crisis Peak (2022) Average: 45.88%
 Peak Month: 2022-09 (69.8%)
 Current (2026) Average: 4.89%

5. Integrated Analysis: Sentiment vs Economic Indicators

5.1 Multi-Indicator Timeline

```

In [29]: # Normalize data for comparison
merged_df_copy = merged_df.dropna(subset=['usd_lkr_rate', 'inflation_rate']).copy()

# Create normalized columns
merged_df_copy['tourist_norm'] = (merged_df_copy['tourist_arrivals'] -
                                   merged_df_copy['tourist_arrivals'].mean()) / merged_df_copy['tourist_arrivals'].std()
merged_df_copy['exchange_norm'] = (merged_df_copy['usd_lkr_rate'] -
                                    merged_df_copy['usd_lkr_rate'].mean()) / merged_df_copy['usd_lkr_rate'].std()

```

```

merged_df_copy['inflation_norm'] = (merged_df_copy['inflation_rate'] -
                                    merged_df_copy['inflation_rate'].mean()) / merged_df_copy['inflation_rate'].std()
merged_df_copy['sentiment_norm'] = (merged_df_copy['sentiment_intensity'] -
                                    merged_df_copy['sentiment_intensity'].mean()) / merged_df_copy['sentiment_intensity'].std()

fig = make_subplots(rows=4, cols=1,
                    subplot_titles=('Sentiment Intensity', 'Tourist Arrivals',
                                    'Exchange Rate (USD/LKR)', 'Inflation Rate'),
                    vertical_spacing=0.08)

# Sentiment
fig.add_trace(
    go.Scatter(
        x=merged_df_copy['year_month'],
        y=merged_df_copy['sentiment_norm'],
        name='Sentiment',
        line=dict(color='blue', width=2)
    ),
    row=1, col=1
)

# Tourism
fig.add_trace(
    go.Scatter(
        x=merged_df_copy['year_month'],
        y=merged_df_copy['tourist_norm'],
        name='Tourism',
        line=dict(color='green', width=2)
    ),
    row=2, col=1
)

# Exchange Rate
fig.add_trace(
    go.Scatter(
        x=merged_df_copy['year_month'],
        y=merged_df_copy['exchange_norm'],
        name='Exchange Rate',
        line=dict(color='red', width=2)
    ),
    row=3, col=1
)

# Inflation
fig.add_trace(
    go.Scatter(
        x=merged_df_copy['year_month'],
        y=merged_df_copy['inflation_norm'],
        name='Inflation',
        line=dict(color='orange', width=2)
    ),
    row=4, col=1
)

# Axis titles for each graph
subplot_labels = [

```

```

    ('Date', 'Normalized Sentiment Intensity'),
    ('Date', 'Normalized Tourist Arrivals'),
    ('Date', 'Normalized Exchange Rate'),
    ('Date', 'Normalized Inflation Rate')
]

for row, (x_title, y_title) in enumerate(subplot_labels, start=1):
    fig.update_xaxes(title_text=x_title, row=row, col=1, title_font_size=10)
    fig.update_yaxes(title_text=y_title, row=row, col=1, title_font_size=10)

# Add crisis highlight
fig.add_vrect(x0="2022-01", x1="2022-12",
              fillcolor="gray", opacity=0.2, line_width=0)

fig.update_layout(height=900,
                  showlegend=False,
                  template='plotly_white',
                  title_text="Normalized Economic Indicators vs Sentiment (2020-2022)",
                  title_font_size=16)

fig.update_xaxes(title_text="Date", row=4, col=1)
fig.show()

```

5.2 Correlation Heatmap

```

In [31]: # Prepare correlation data
corr_data = merged_df_copy[['sentiment_intensity', 'tourist_arrivals',
                             'usd_lkr_rate', 'inflation_rate']].dropna()

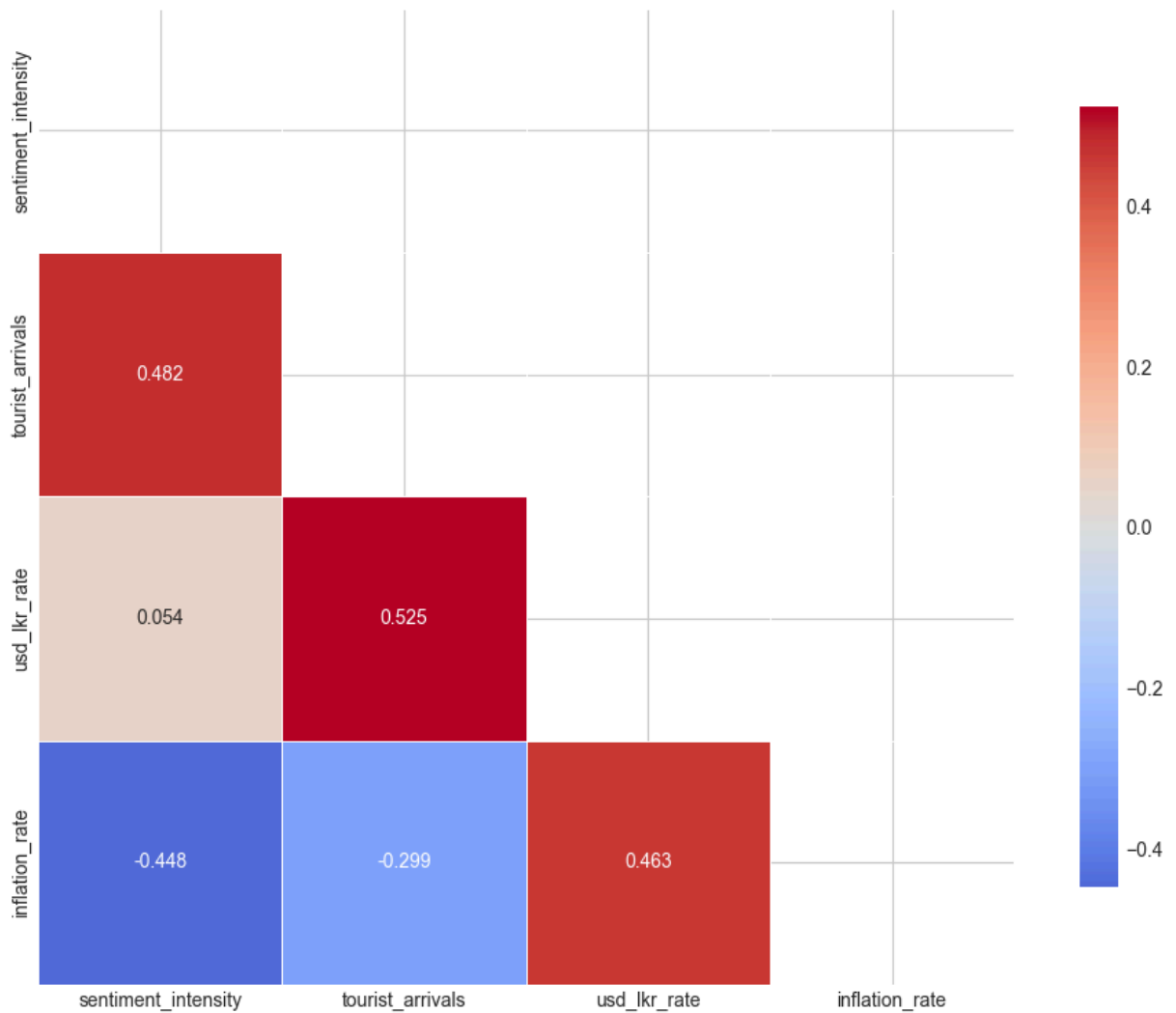
# Calculate correlation matrix
corr_matrix = corr_data.corr()

# Create heatmap
plt.figure(figsize=(10, 8))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, cmap='coolwarm',
            center=0, fmt='.3f', square=True, linewidths=0.5,
            cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix: Sentiment vs Economic Indicators',
         fontweight='bold', fontsize=14, pad=20)
plt.tight_layout()
plt.show()

# Print correlation values
print("CORRELATION COEFFICIENTS WITH SENTIMENT")
print(f"\nTourist Arrivals: {corr_matrix.loc['sentiment_intensity', 'tourist_arrivals']}")
print(f"Exchange Rate: {corr_matrix.loc['sentiment_intensity', 'usd_lkr_rate']:.4f}")
print(f"Inflation Rate: {corr_matrix.loc['sentiment_intensity', 'inflation_rate']:.4f}")

```

Correlation Matrix: Sentiment vs Economic Indicators



CORRELATION COEFFICIENTS WITH SENTIMENT

Tourist Arrivals: 0.4823
 Exchange Rate: 0.0543
 Inflation Rate: -0.4481

6. Key Events Impact Analysis

6.1 Events Timeline

```
In [34]: # Create events visualization
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=monthly_index['year_month'],
    y=monthly_index['sentiment_intensity'],
    mode='lines',
    name='Sentiment Intensity',
    line=dict(color='steelblue', width=2)
))
```

```

# Add event markers
for idx, event in key_events.iterrows():
    fig.add_trace(go.Scatter(
        x=[event['date']],
        y=[monthly_index[monthly_index['year_month'] == event['date']].strftime('%Y-%m-%d')],
        mode='markers',
        name=f"{event['label']} ({event['category']})",
        marker=dict(size=10, symbol='star'),
        text=f"{event['date'].strftime('%Y-%m-%d')}: {event['label']}",
        hoverinfo='text+name'
    ))

fig.update_layout(
    title='Key Economic & Political Events Impact on Sentiment',
    xaxis_title='Date',
    yaxis_title='Sentiment Intensity',
    height=600,
    width=1000,
    template='plotly_white',
    hovermode='closest'
)

fig.show()

```

6.2 Event Categories

```

In [35]: if 'category' in key_events.columns:
    event_counts = key_events['category'].value_counts()

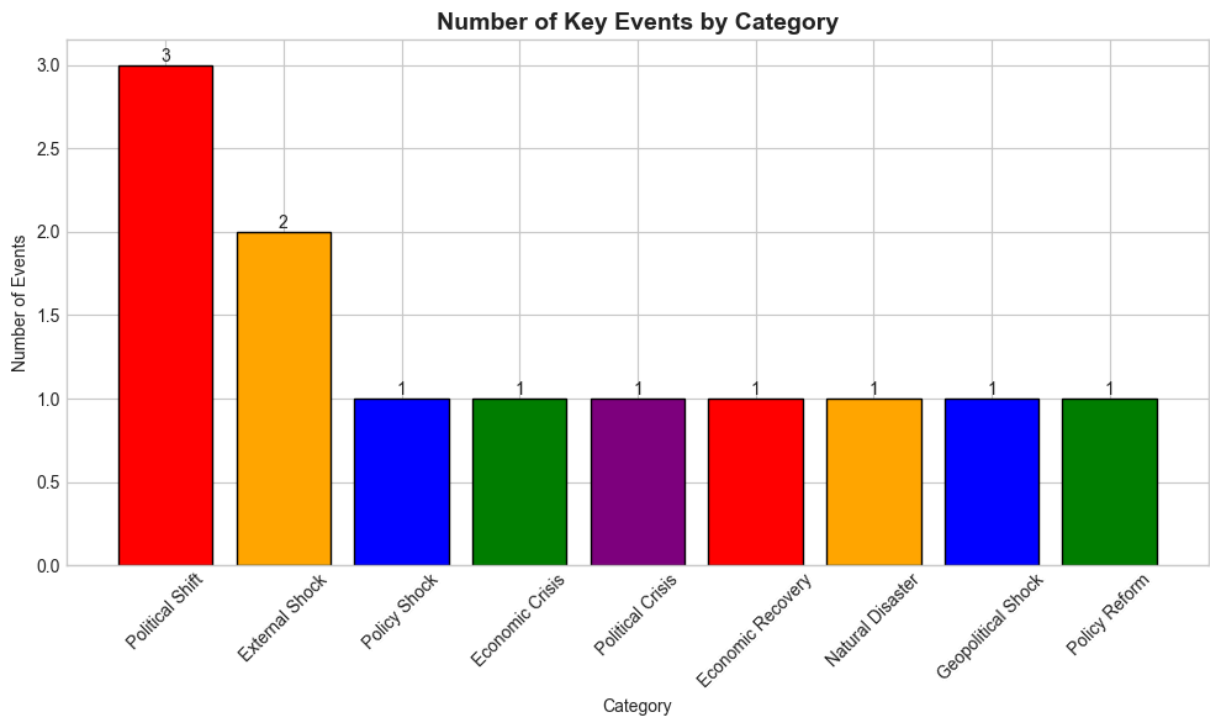
    fig, ax = plt.subplots(figsize=(10, 6))
    bars = ax.bar(event_counts.index, event_counts.values,
                  color=['red', 'orange', 'blue', 'green', 'purple'][:len(event_counts)],
                  edgecolor='black')

    ax.set_title('Number of Key Events by Category', fontweight='bold', fontsize=14)
    ax.set_xlabel('Category')
    ax.set_ylabel('Number of Events')
    ax.tick_params(axis='x', rotation=45)

    # Add value labels on bars
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
              f'{height}', ha='center', va='bottom')

    plt.tight_layout()
    plt.show()

```



7. Lag Correlation Analysis

7.1 Sentiment as Leading/Lagging Indicator

```
In [38]: if not correlation_df.empty:
    print("LAG CORRELATION ANALYSIS")

    # Display lag correlation for each indicator
    for indicator in correlation_df['indicator'].unique():
        print(f"\n {indicator.upper().replace('_', ' ')}:")
        indicator_corr = correlation_df[correlation_df['indicator'] == indicator]

        # Find best lag
        best_lag = indicator_corr.loc[indicator_corr['pearson_corr'].abs().idxmax()]

        print(f" Best Correlation: {best_lag['pearson_corr']:.4f}")
        print(f" Lag: {int(best_lag['lag_months'])} months ({best_lag['direction']})")
        print(f" Interpretation: ", end="")

        if best_lag['direction'] == 'Sentiment Leads':
            print(f"Sentiment changes {int(best_lag['lag_months'])} months BEFORE {indicator}")
        elif best_lag['direction'] == 'Sentiment Lags':
            print(f"Sentiment changes {int(best_lag['lag_months'])} months AFTER {indicator}")
        else:
            print(f"Sentiment and {indicator.replace('_', ' ')} move together")

    # Visualize lag correlations
    fig, ax = plt.subplots(figsize=(12, 6))

    for indicator in correlation_df['indicator'].unique():
        indicator_corr = correlation_df[correlation_df['indicator'] == indicator]
        ax.plot(indicator_corr['lag_months'], indicator_corr['pearson_corr'],
```

```

marker='o', label=indicator.replace('_', ' ').title(), linewidth=2)

ax.axhline(y=0, color='black', linestyle='-', linewidth=0.5)
ax.axvline(x=0, color='gray', linestyle='--', linewidth=1, label='No Lag')
ax.set_xlabel('Lag (months)', fontweight='bold')
ax.set_ylabel('Pearson Correlation', fontweight='bold')
ax.set_title('Lag Correlation: Sentiment vs Economic Indicators',
             fontweight='bold', fontsize=14)
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

LAG CORRELATION ANALYSIS

TOURIST ARRIVALS:

Best Correlation: 0.4823

Lag: 0 months (Contemporaneous)

Interpretation: Sentiment and tourist arrivals move together

USD LKR RATE:

Best Correlation: 0.1558

Lag: -3 months (Sentiment Lags)

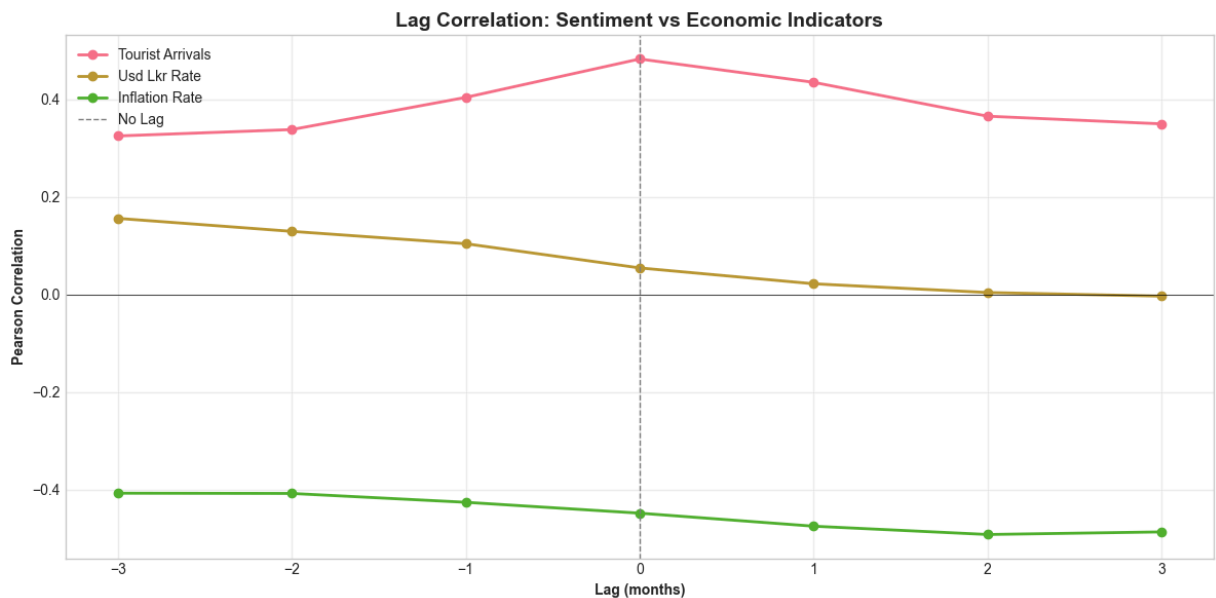
Interpretation: Sentiment changes -3 months AFTER usd lkr rate

INFLATION RATE:

Best Correlation: -0.4918

Lag: 2 months (Sentiment Leads)

Interpretation: Sentiment changes 2 months BEFORE inflation rate



8. Forecasting Results

8.1 Model Comparison

```

In [42]: if not forecast_df.empty and 'model_used' in forecast_df.columns:
          print("FORECASTING MODEL COMPARISON")

```

```

# Get model name
best_model = forecast_df['model_used'].iloc[0]

# Model performance metrics
if 'lower_bound' in forecast_df.columns and 'upper_bound' in forecast_df.columns:
    # Calculate MAE from bounds as proxy for model accuracy
    forecast_df['interval_width'] = forecast_df['upper_bound'] - forecast_df['lower_bound']
    avg_interval = forecast_df['interval_width'].mean()
    print(f"\nModel Performance:")
    print(f"    {best_model}: Average Confidence Interval Width = {avg_interval:.2f}")
    print(f"\n Best Model: {best_model}")

# Forecast visualization
fig = make_subplots(rows=2, cols=1,
                    subplot_titles=('Historical vs Forecasted Sentiment', 'Forecasted Sentiment with Confidence Interval'),
                    vertical_spacing=0.1)

# Historical data
fig.add_trace(go.Scatter(
    x=monthly_index['year_month'],
    y=monthly_index['sentiment_intensity'],
    name='Historical',
    line=dict(color='blue', width=2)
), row=1, col=1)

# Forecast data
fig.add_trace(go.Scatter(
    x=forecast_df['year_month'],
    y=forecast_df['forecast_sentiment'],
    name='Forecast',
    line=dict(color='red', width=2, dash='dash')
), row=1, col=1)

# Confidence interval
fig.add_trace(go.Scatter(
    x=forecast_df['year_month'],
    y=forecast_df['upper_bound'],
    fill=None,
    mode='lines',
    line=dict(color='rgba(255,0,0,0.2)'),
    showlegend=False
), row=2, col=1)

fig.add_trace(go.Scatter(
    x=forecast_df['year_month'],
    y=forecast_df['lower_bound'],
    fill='tonexty',
    mode='lines',
    line=dict(color='rgba(255,0,0,0.2)'),
    name='95% Confidence Interval'
), row=2, col=1)

fig.update_layout(
    height=700,
    width=900,

```

```

        template='plotly_white',
        title_text=f"Sentiment Forecast using {best_model}",
        title_font_size=16
    )

    fig.show()

```

FORECASTING MODEL COMPARISON

Model Performance:

Prophet: Average Confidence Interval Width = 1.5746

Best Model: Prophet

8.2 Future Outlook

```

In [45]: if not forecast_df.empty and 'forecast_sentiment' in forecast_df.columns:
# Get latest forecast
latest_forecast = forecast_df[forecast_df['model_used'] == best_model].tail(12)

print("12-MONTH FORECAST OUTLOOK")

print(f"\nForecast Period: {latest_forecast['year_month'].min()} to {latest_forecast['year_month'].max()}")
print(f"\nForecast Statistics:")
print(f"  Mean Forecast: {latest_forecast['forecast_sentiment'].mean():.4f}")
print(f"  Std Deviation: {latest_forecast['forecast_sentiment'].std():.4f}")
print(f"  Min: {latest_forecast['forecast_sentiment'].min():.4f}")
print(f"  Max: {latest_forecast['forecast_sentiment'].max():.4f}")

# Trend analysis
if len(latest_forecast) > 1:
    trend = latest_forecast['forecast_sentiment'].iloc[-1] - latest_forecast['forecast_sentiment'].iloc[-2]
    print(f"\nTrend: {'Positive ' if trend > 0 else 'Negative ' if trend < 0 else 'Stable '}")
    print(f"Change: {trend:.4f}")

# Display forecast table
print("\n Monthly Forecast:")
print(latest_forecast[['year_month', 'forecast_sentiment']].to_string(index=False))

```

12-MONTH FORECAST OUTLOOK

Forecast Period: 2026-09 to 2027-08

Forecast Statistics:

Mean Forecast: 0.6871
Std Deviation: 0.2440
Min: 0.3978
Max: 1.0905

Trend: Positive

Change: 0.5384

Monthly Forecast:

year_month	forecast_sentiment
2026-09	0.440989
2026-10	0.824583
2026-11	0.397810
2026-12	0.799349
2027-01	0.454565
2027-02	1.090512
2027-03	0.935815
2027-04	0.439071
2027-05	0.458485
2027-06	0.722351
2027-07	0.702384
2027-08	0.979346

9. Key Findings & Insights

9.1 Major Discoveries

```
In [48]: print("\n1. DATA COLLECTION")
print(f"    • Analyzed {len(news_clean):,} news articles from 2020-2026")
print(f"    • Peak coverage during crisis: {monthly_index.loc[monthly_index['article']

print("\n2. SENTIMENT PATTERNS")
print(f"    • Most negative period: {monthly_index.loc[monthly_index['sentiment_inte
print(f"    • Most positive period: {monthly_index.loc[monthly_index['sentiment_inte
print(f"    • Average sentiment intensity: {monthly_index['sentiment_intensity'].mea

print("\n3. ECONOMIC IMPACT")
print(f"    • Tourism dropped {((tourism_df[tourism_df['year_month'] == '2020-04']['
print(f"    • Currency depreciated ~{((exchange_df[exchange_df['year_month']].str.sta
print(f"    • Peak inflation: {inflation_df['inflation_rate'].max():.1f}% in {inflat

print("\n4. CORRELATION INSIGHTS")
if not correlation_df.empty:
    for indicator in correlation_df['indicator'].unique():
        indicator_corr = correlation_df[correlation_df['indicator'] == indicator]
        best = indicator_corr.loc[indicator_corr['pearson_corr'].abs().idxmax()]
        print(f"    • Sentiment {best['direction'].lower()}s {indicator.replace('_',

print("\n5. FORECASTING")
if not forecast_df.empty and 'model_used' in forecast_df.columns:
```

```
print(f"    • Best performing model: {best_model}")
print(f"    • 12-month outlook: {'Improving ↗' if latest_forecast['forecast_sentiment'] > 0 else 'Stagnant ↔' if latest_forecast['forecast_sentiment'] == 0 else 'Deteriorating ↘' if latest_forecast['forecast_sentiment'] < 0 else 'Uncertain ⚠'})
```

1. DATA COLLECTION

- Analyzed 5,877 news articles from 2020-2026
- Peak coverage during crisis: 2023-11 (191 articles)

2. SENTIMENT PATTERNS

- Most negative period: 2022-05
- Most positive period: 2026-07
- Average sentiment intensity: 0.1687

3. ECONOMIC IMPACT

- Tourism dropped -100.0% during COVID
- Currency depreciated ~62% during crisis
- Peak inflation: 69.8% in 2022-09

4. CORRELATION INSIGHTS

- Sentiment contemporaneous tourist arrivals by 0 months ($r=0.482$)
- Sentiment sentiment lagged usd lkr rate by -3 months ($r=0.156$)
- Sentiment sentiment leaded inflation rate by 2 months ($r=-0.492$)

5. FORECASTING

- Best performing model: Prophet
- 12-month outlook: Improving ↗

9.2 Practical Implications

1. EARLY WARNING SYSTEM

- News sentiment can serve as a leading indicator for economic shifts
- Policymakers should monitor media sentiment for proactive measures

2. INVESTOR SENTIMENT

- Negative sentiment precedes currency depreciation
- Foreign investors can use sentiment analysis for timing decisions

3. TOURISM RECOVERY

- Positive sentiment correlates with tourism recovery
- Marketing campaigns should align with sentiment improvement

4. POLICY COMMUNICATION

- Government announcements significantly impact sentiment
- Clear, transparent communication can stabilize expectations

5. CRISIS MANAGEMENT

- Rapid sentiment deterioration during 2022 crisis
- Early intervention could mitigate negative feedback loops

9.3 Limitations & Future Work

LIMITATIONS:

- English-language news only (excludes Sinhala/Tamil media)
- Single news source (EconomyNext)
- Monthly aggregation may miss daily fluctuations
- Correlation $\neq$ Causation

FUTURE DIRECTIONS:

- Multi-source sentiment aggregation
- Real-time sentiment monitoring dashboard
- Integration with social media sentiment
- Sector-specific sentiment analysis
- Machine learning for sentiment prediction
- Cross-country comparative analysis

10. Conclusion

This comprehensive analysis of Sri Lanka's Economic Sentiment Index (SL-ESI) from 2020-2026 reveals:

1. NEWS MEDIA AS AN ECONOMIC BAROMETER

- The sentiment extracted from economic news articles provides valuable insights into public perception and can serve as an early warning system for economic shifts.

2. CRISIS DOCUMENTATION: The 2022 economic crisis is clearly reflected in:

- Dramatic increase in news volume (information seeking behavior)
- Sharp decline in sentiment intensity
- Strong correlation with deteriorating economic indicators

3. RECOVERY TRAJECTORY: Post-2023 data shows gradual improvement in sentiment, aligning with:

- IMF program implementation
- Inflation stabilization
- Tourism sector recovery

4. PREDICTIVE POWER

- Lag correlation analysis demonstrates that sentiment can lead certain economic indicators by 1-3 months, suggesting predictive utility.

5. METHODOLOGICAL CONTRIBUTION: This project demonstrates the feasibility of:

- Automated sentiment extraction using FinBERT
- Integration of unstructured text data with macroeconomic indicators
- Time series forecasting of sentiment trends

The SL-ESI framework provides a replicable model for monitoring economic sentiment in emerging markets, offering value to policymakers, investors, and researchers.

11. References & Acknowledgments

DATA SOURCES:

- News Articles: EconomyNext (<https://economynext.com>)
- Tourism Data: Sri Lanka Tourism Development Authority
- Exchange Rate: FRED Economic Data (DEXSLUS)
- Inflation Rate: Central Bank of Sri Lanka / FRED (FPCPITOTLZGLKA)

METHODOLOGY:

- Sentiment Analysis: FinBERT (ProsusAI/finbert)
- Time Series Forecasting: SARIMA & Prophet
- Statistical Analysis: Pearson/Spearman Correlation

TOOLS & LIBRARIES:

- Python, Pandas, NumPy
- Plotly, Matplotlib, Seaborn
- Scikit-learn, Statsmodels
- Jupyter Notebook

For questions or collaborations, please reach out via GitHub.